

Speed Rail dogfood — step by step

This is the slow walk-through. If anything below doesn't match what you see on screen, stop and shout — don't guess past it.

One-time setup is steps A–F. Daily use is just step G onwards. After the first run you can ignore A–F unless something changes.

A. Set up the database (Neon)

You only do this once. Neon is the company that runs the database that holds all the ERP's data.

1. Open a browser, go to <https://neon.tech>, click **Sign up** (top right). Use your gilbertrolfe@gmail.com account — the GitHub or Google sign-in button is fastest.
2. After sign-up Neon asks for a project. Set:
 - **Project name:** `back-bar`
 - **Database name:** leave as default (`neondb`)
 - **Region:** pick the one closest to London (usually labelled `eu-west` or `EU (Frankfurt)` / `EU (London)`)
 - Click **Create**.
3. Once the project is created, Neon shows a **Connection string** box at the top of the dashboard. There's usually a tab labelled **Pooled connection** — pick that one.
4. Click the **copy** icon next to the connection string. It looks like: `postgresql://neondb_owner:abc123XYZ@ep-cool-name-123-pooler.eu-west-2.aws.neon.tech/neondb?sslmode=require` Keep this string handy for step C. **Treat it like a password.**

■ You're done with Neon's website until later.

B. Open a terminal in the project

The "terminal" is a window where you type commands.

1. Open the **Terminal** app on your Mac (⌘+Space, type `Terminal`, hit Enter).
 2. Speed Rail lives on a feature branch in a *worktree* (a separate copy of the repo) until it's merged. Move into that worktree: `cd ~/code/myattsflds/back-bar/.claude/worktrees/xenodochial-heisenberg-d6e7c4` (Press Enter after every command in this guide.)
- > **Don't use** `~/code/myattsflds/back-bar` **directly** for Speed Rail — that's the main checkout and doesn't have the ERP code yet. After slice 2 lands as a draft PR, the worktree path may change; if it does, I'll tell you.

3. Confirm you're in the right place — type: `ls` You should see `package.json`, `src`, `docs`, `drizzle`, `seed` and so on. If you don't see `drizzle` and `seed`, you're in the wrong folder.

C. Tell Back Bar where the database lives

1. In your terminal, run: `cp .env.example .env.local` This makes a private copy of the template settings file. The `.env.local` file is your secrets file — never commit it, never share it.

> If you already had a `.env.local` in `~/code/myattsfIELDS/back-bar` (the main folder), it doesn't carry over — the worktree has its own. Easiest path: pull all Vercel values in one shot in the next step, then just add the three Neon-related lines.

2. Pull all the existing Vercel-managed env values (auth password, QuickBooks tokens, etc.) into the new file: `vercel env pull .env.local` This gives you a starting point with the QB and auth lines already filled in. (If `vercel` isn't installed: `npm i -g vercel` then run the command. The first time you run any `vercel` command it'll ask you to log in and link the project — say yes to both.)

> **If you see** Your codebase isn't linked to a project on Vercel: this is a worktree-only thing — the main repo is linked but the worktree isn't yet. Quickest fix is to copy the link folder from the main repo: `> cp -r ~/code/myattsfIELDS/back-bar/.vercel .` > Then re-run `vercel env pull .env.local`. (Or run `vercel link` and answer Yes to everything, picking `back-bar` as the project.)

3. Open `.env.local` in a text editor. Either:

- In Terminal: `open .env.local` (opens in TextEdit), or
- In Finder: navigate to the worktree folder and double-click the file. (If you don't see it, it's hidden — press `⌘+Shift+.` in Finder to show hidden files.)

4. Add three new lines at the bottom (or fill them in if `vercel env pull` already inserted blank versions): `DATABASE_URL=postgresql://neondb_owner:abc123XYZ@ep-cool-name-123-pooler.eu-west-2.aws.neon.tech/neondb?sslmode=require`
`SPEED_RAIL_ENABLED=1` `NEXT_PUBLIC_SPEED_RAIL_ENABLED=1` The `DATABASE_URL` value is the Neon string you copied in step A4. The other two are literally `1`.

5. Save and close the file. The QuickBooks and auth lines stay alongside the Neon ones — env files just hold key=value pairs and don't care which group is which.

D. Create the database tables

This builds the empty cabinets that data goes into.

In your terminal, run:

```
npm run db:migrate
```

You should see lines like `[✓] migration applied`. If you see a red `error` instead, copy the error and bring it back to chat.

E. Load the starter data

This puts a 20-row starter list of components and the three system settings (wastage, labour rate, next serial number) into the empty database.

In your terminal, run:

```
npm run db:seed
```

You should see something like:

```
✓ Seeded 3 system settings
✓ Components: 20 inserted, 0 already present.
Done. Next: open /erp once SPEED_RAIL_ENABLED=1 is set.
```

If you ever want to re-run this, it's safe — it won't double-up rows.

F. Start the dev server

In your terminal, run:

```
npm run dev
```

Wait for a line that looks like `✓ Ready on http://localhost:3000`. Leave the terminal window open — closing it stops the server.

G. Open the ERP in your browser

1. Open **`http://localhost:3000`** in a browser.
 2. You'll see the login screen. Type the Back Bar password and hit **Enter**.
 3. In the top navigation bar, you should now see **ERP** as one of the links. Click it. You're in.
-

H. The smoke test (the bit that proves it actually works)

Do this in order. After every step, glance at the screen and confirm what's described.

1. Read the landing page

- You should see the **ERP** title, the slice roadmap, and three module rows under "Slice 1 — Foundations".
- Each row has a count: Suppliers **0**, Components **20**, Settings **3**. (Components = 20 because the seed loaded them.)

2. Add a supplier

- Click **Suppliers** (in the sub-nav under "ERP").
- The page says "0 on file" and shows an empty state. Click **Add the first one**.
- Fill in:
 - Name: **Bimber**
 - Contact email: anything
 - Payment terms: **30 days**
 - Default currency: leave as **GBP**
- Click **Create supplier**.
- You should land back on the suppliers list, and Bimber should be there as **Active**.

3. Edit and deactivate a supplier

- Click Bimber's name.
- Change anything (e.g. add a phone number) and click **Save changes**. The list reloads, the change should stick.
- Click Bimber again, scroll to the bottom, click **Deactivate supplier**. The status badge should flip to **Inactive**, the link should still work.
- Reactivate it from the same button.

4. Edit a component

- Click **Components** in the sub-nav.
- You should see the 20 seeded components, grouped by type (Ingredient, Dry good, Packaging).
- Click **Gin (in-house)**.
- Scroll down — under **Price history** you should see a single row from the seed (£0.0225, source: manual).
- Change the unit cost to **0.0250**. Optionally set the default supplier to Bimber. Click **Save changes**.
- Open Gin again. You should now see **two rows** in price history — the old one and a new one. The list page should show £0.0250 and today's date in the "Set" column.

5. Edit settings

- Click **Settings** in the sub-nav.
- Change the wastage to **2.5**, click **Save settings**.

- Reload the page. The number should still say 2.5. Each field should now show "updated [today's date]" above it.
- Set wastage back to 2.

6. Confirm the flag actually gates the routes

- Stop the dev server: in the terminal where it's running, press **Ctrl+C**.
 - Open `.env.local`, change `SPEED_RAIL_ENABLED=1` to `SPEED_RAIL_ENABLED=` (blank). Save.
 - Run `npm run dev` again.
 - Visit `http://localhost:3000/erp`. You should see a **404 Not Found** page.
 - Also notice the **ERP** nav link still shows because the *public* flag is still on.
 - Now also blank out `NEXT_PUBLIC_SPEED_RAIL_ENABLED=`, restart the server. The ERP nav link should be gone.
 - Set both back to 1 when you're done — restart the server.
-

What to flag back to me

After dogfooding, tell me:

- Anything that didn't behave as the smoke test described.
- Any field on the supplier or component form that you'd want renamed, hidden, or split.
- Whether the components-by-type grouping makes sense or if you'd rather see them flat.
- Anything that took more clicks than it should.
- Any wording that confused you.

When you're happy with it, say "go on slice 2" and I'll start the Inbounds slice.

Common things that can go wrong

What you see	What to do
<code>DATABASE_URL</code> is not set	Step C didn't save. Re-open <code>.env.local</code> , check the line is there, save again, restart <code>npm run dev</code> .
password authentication failed after <code>npm run db:migrate</code>	The Neon connection string has a typo. Re-copy it from Neon (use the pooled one), repaste into <code>.env.local</code> , save, retry.
<code>npm run dev</code> shows address already in use	Another <code>npm run dev</code> is already running somewhere. Either close that terminal or change the port: <code>PORT=3001 npm run dev</code> .
<code>/erp</code> shows 404	Either the flag is off (check <code>.env.local</code>) or you didn't restart the dev server after editing <code>.env.local</code> . Always restart after env changes.
The "ERP" nav link is missing	<code>NEXT_PUBLIC_SPEED_RAIL_ENABLED=1</code> isn't set, or you didn't restart the server.